Assignment #1

Due: February 12th 2026

1. Introduction

In this assignment, you will model the kinematics of a UR5 from Universal Robot illustrated in Figure 1, simulate and animate its motion. You will modify and submit existing C++ code to generate a trajectory of the robot. This assignment should also be used to familiarize yourself with Linux and ROS. You will use this environment throughout the semester, so it is a good investment of your time to [try a few tutorials] and understand how to use ROS topics to read/write information about the robot.

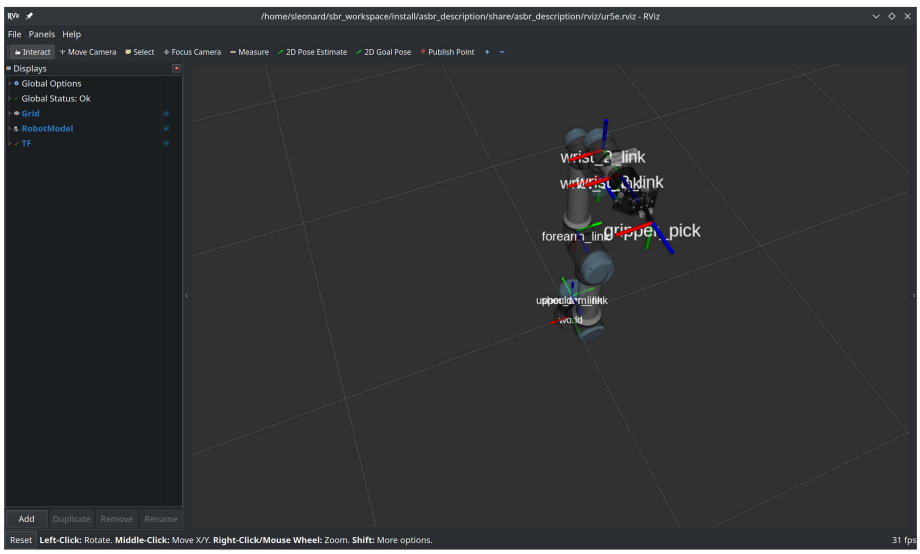

Figure 1: The UR5 robot with Robotiq gripper.

1.1 Deliverables

1. The file assignment1.tgz package that evaluates the forward kinematics and the manipulator Jacobian of the UR5

2. This assignment is worth 9 points, but you will be required to demonstrate the trajectory generator on a real UR5 at the end of the semester to earn the 10th point. Details about demos will be provided in March/April.

2. Installation and Configuration of Ubuntu and ROS

If you do not have a computer with Ubuntu and ROS, you have the following options.

2.1 Install Ubuntu on a computer (standalone or dual boot)

If you have a spare computer and you want to learn how to install Linux (Ubuntu) you can install Ubuntu 22.04. Installation instructions are available at https://ubuntu.com/download/desktop. This requires more effort, but it is a worthwhile

experience (I recommend *all* MSE robotics students to learn how to manage a Linux computer). A standalone installation will take over your entire computer. A dual-boot installation will keep your current operating system (OS) and install Ubuntu on the same hard drive (be very careful not to wipe your current OS!). Each time you start your computer you will be asked to select which OS you want to boot.

Your next step will be to install and configure ROS. We use ROS2 Humble. ROS installation instructions are provided at the following URL https://docs.ros.org/en/humble/Installation.html.

2.2 Virtual Machine

If you use an Apple or Windows computer and want to keep your computer "as is", you can install Ubuntu as a virtual machine with VirtualBox or VMware. This will enable you to run Ubuntu (guest OS) inside your current operating system (host OS). You will need a computer with a sizeable amount of memory (8GB+) since your computer will run two OS at the same time and a hard drive with at least 20 GB of free space.

Once a VM is installed, you can install Ubuntu like you would with a regular computer by following the instructions here https://www.ubuntu.com/download/desktop.

Note that VMs are not perfect. Students have experienced problems with certain graphical applications and simulating cameras in upcoming assignments as well as slow performances. In my experience, most of these issues can be addressed but they require a small amount of work and patience.

2.3 Docker

If you are familiar with Docker, there are images available, and instructions can be found here. I am not familiar with Docker but some students have used this in the past.

2.4 Wyman 170

You have access to the computers in Wyman170. You will need your JCard to access the lab and the building after regular hours. Currently there are 12 workstations in 170. These computers have all the software and packages needed for this course (ROS, Matlab, etc.). You must use your JHED ID and password to login. All Wyman 170 computers are fully configured, and you will not need to install any packages (i.e. no need to run any "sudo" commands). If you use these computer, you must follow these instructions to configure your account. Note that student accounts have a 20GB quota and will be removed at the end of the semester. These workstations have 10 3.7GHz cores and 70GB of RAM but the student accounts are networked which slows down the process of building packages significantly.

2.5 Familiarize yourself with ROS

One important aspect of this assignment is to become familiar with Linux and ROS. If you are not familiar with ROS, the following tutorials are highly recommended https://docs.ros.org/en/humble/Tutorials.html. Especially those about nodes, publisher/subscriber, topics and tf (transformation manager). These tutorials are a very good investment of your time for robotics courses.

If you work on your own computer, you will need to install several additional

packages for this assignment; you can install them by following commands (you do not need to do these commands on the Wyman computers).

```
$ sudo apt-get update
$ sudo apt-get install  ros-humble-control*
                        ros-humble-ros2-control*
                        ros-humble-moveit*
                        ros-humble-robotiq*
                        ros-humble-ign-ros2-control
                        ros-humble-ros-ign*
                        ros-humble-joint-state-publisher-gui
                        ros-humble-kinematics-interface-kdl
                        ros-humble-rqt-joint-trajectory-controller
                        python3-colcon*
```

3. Installation of the SBR package

j

Download and run the following script. It will create your ROS workspace for this course and compile the dependencies.

```
$ wget www.cs.jhu.edu/~sleonard/sbr-a1.sh
$ chmod 755 sbr-a1.sh
$ ./sbr-a1.sh
```

3.1 GUI animation of the robot

First, start a terminal (CTR+ALT+T) go the SBR workspace and start a simulation of the robot

```
$ cd ~/sbr_workspace
$ source install/setup.bash
$ ros2 launch assignment1 assignment1.launch.py
```

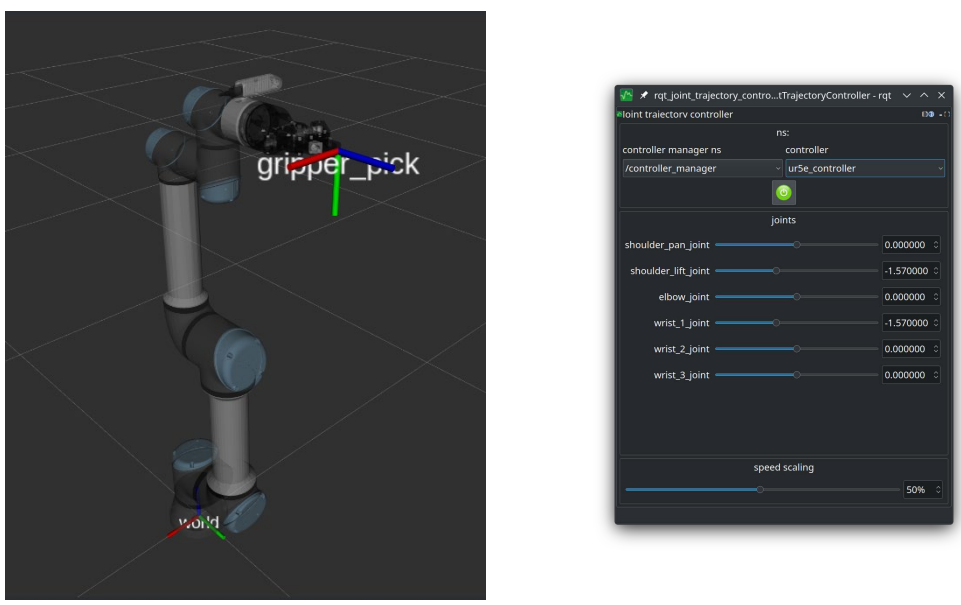

Figure 2: The Joint Trajectory Controller and Rviz.

You should see the two windows illustrated in Figure 2. Ensure that the controller is selected from the drop-down menu and the controller is enabled (toggle button in the middle). Feel free to move the robot and its coordinate frames by using the sliders. Try the values shown in the figure to bring the robot straight up. You can also familiarize yourself with Rviz by expanding the drop-down menus in the “Displays” pane (on the left). The “TF”

element you will be able to monitor the position and orientation of each coordinate frame as you move the robot (something that will be useful to validate your results). With this tool, you can literally determine if your calculations are correct. So technically there is no excuse for a mistake.

Once you are done, list the nodes that were launched by typing the following command in a terminal

```
$ ros2 node list
     /controller_manager
     /get_controller_managers_node
     /gz_ros2_control
     /robot_state_publisher
     /robotiq_gripper_controller
     /robotiq_joint_state_broadcaster
     /ros_gz_bridge
     /rqt_gui_py_node_80127
     /rqt_joint_trajectory_controller
     /rviz2
     /static_transform_publisher_RTBx2f4mxP6U7iLi
     /trajectory
     /transform_listener_impl_63d338b1d250
     /ur5e_controller
     /ur_joint_state_broadcaster
```

We will focus on the `rqt_joint_trajectory_controller` and the `robot_state_publisher` nodes.

3.2 rqt_joint_trajectory_controller

The `rqt_joint_tracjectory_controller` node is the node associated with the small window that publishes joint values. Each time you move a slider, a new command of joint positions is issued to the robot. You can find more information about this node by typing

```
$ ros2 node info /rqt_joint_trajectory_controller
     Publishers:
      /joint_trajectory_controller/joint_trajectory:
trajectory_msgs/msg/JointTrajectory
```

The GUI node publishes messages of the type `JointTrajectory` on the topic `/ur5e_controller/joint_trajectory`. You can print the values of the joint to the screen by typing the command

```
$ ros2 topic echo /ur5e_controller/joint_trajectory

joint_names:
- shoulder_pan_joint
- shoulder_lift_joint
- elbow_joint
- wrist_1_joint
- wrist_2_joint
- wrist_3_joint
points:
- positions:
  - -0.0
  - -1.57
  - 0.0
```

```
  - -1.57
  - -0.0
  - -0.0
  velocities: []
  accelerations: []
  effort: []
  time_from_start:
    sec: 1
   nanosec: 0
```

Moving the sliders will print out the current command of joints position. The message printed above indicates the six joint names and the command position for each joint (values are in radian).

3.3 robot_state_publisher

The node `robot_state_publisher` subscribes to the topic `/joint_states` that contains the *measured* (as opposed to the *commands*) joint angles and computes the forward kinematics for each link. This enables to transform all the coordinate frames and 3D geometries so they can be viewed at their current position and orientation. You can see information about `robot_state_publisher` by typing (some output is omitted)

```
$ ros2 node info /robot_state_publisher
       Subscribers:
         /clock: rosgraph_msgs/msg/Clock
         /joint_states: sensor_msgs/msg/JointState
         /parameter_events: rcl_interfaces/msg/ParameterEvent
       Publishers:
         /parameter_events: rcl_interfaces/msg/ParameterEvent
         /robot_description: std_msgs/msg/String
         /rosout: rcl_interfaces/msg/Log
         /tf: tf2_msgs/msg/TFMessage
         /tf_static: tf2_msgs/msg/TFMessage
```

The output indicates that the node subscribes to `/joint_states` to read the *measured* joint values (that topic is published by the simulator), computes the forward kinematics, and publishes the resulting transformations on the topic `/tf`.

If you want to know the position and orientation of the gripper (named `gripper_pick`) with respect to the coordinate frame of the world (name `world`) you can query the transformation manager by typing

```
$ ros2 run tf2_ros tf2_echo world gripper_pick
     - Translation: [0.001, 0.331, 1.001]
     - Rotation: in Quaternion [-0.707, 0.001, 0.001, 0.707]
     - Rotation: in RPY (radian) [-1.571, 0.002, -0.000]
     - Rotation: in RPY (degree) [-90.000, 0.091, -0.000]
     - Matrix:
       1.000 -0.002  0.000  0.001
      -0.000 -0.000  1.000  0.331
      -0.002 -1.000 -0.000  1.001
       0.000  0.000  0.000  1.000
```

The output indicates that the coordinate frame `gripper_pick` is at the position [0.001, 0.331, 1.001] and its orientation is given by the quaternion [-0.707, 0.001, 0.001, 0.707]. You can also find the position/orientation of the forearm with respect to its predecessor frame (upper arm) with

```
$ ros2 run tf2_ros tf2_echo upper_arm_link forearm_link
```

You can also see the position/orientation of each link in Rviz by expanding the "TF->Frames" display (Rviz also gives better precision than tf2_echo). The information provided by TF is very useful to debug your forward kinematics because you can always validate if your solution is correct by comparing your values to TF.

4. Forward Kinematics of the UR5 and its Inverse

This assignment uses the six joints $\boldsymbol{q} = [\ q_1\ q_2\ q_3\ q_4\ q_5\ q_6\ ]^T$ (two joints in the shoulder, the elbow and three in the wrist) of the UR5. Your first task is to derive the forward kinematics of the UR5. Specifically, you must compute the 3D rotation and 3D position of the coordinate frame associated with the `gripper_pick` frame with respect to the world frame as a function of the joint angles $q_1, \ldots, q_6$. All six joints are revolute, and the joints are associated with the following coordinate frames:

Joint Name	Link Name
shoulder_pan_joint	shoulder_link
shoulder_lift_joint	upper_arm_link
elbow_joint	forearm_link
wrist_1_joint	wrist_1_link
wrist_2_joint	wrist_2_link
wrist_2_joint	wrist_3_link

You need to find the transformation between consecutive frames ${}^{i}E_j$. All joints rotate about a "Z" axis and you can use `tf2_echo` (or Rviz) to determine all the degrees of freedom (DoF) of each ${}^{i}E_j$. You can validate your results by changing value of a joint and comparing to `tf2_echo`'s results (if you use Matlab, the "subs" command can substitute values to symbols). Once you are done, you can combine all the transformations to produce the forward kinematics. If you use Matlab you can use the command "simplify" on each ${}^{i}E_j$ and the resulting FK. The command `tf2_echo` is a tool that you can use to find any mistakes in your ${}^{i}E_j$ calculations, no excuse for mistakes.

5. Inverse Jacobian Trajectory Generator

We want to move the robot without the node `rqt_joint_trajectory_controller` by replacing the sliders with a trajectory generator that automatically and gradually moves the robot from its current position to the 3D coordinates that you specify on a command line. For example, if you want to move the gripper from its current Cartesian coordinates to a point (for example ${}^{W}P = [0.5\ 0.5\ 0.2]^T$) in the world coordinate frame, the robot will need to move its joints gradually toward that goal.

For this, we will implement a "resolved motion rate controller" that uses the manipulator Jacobian to move the gripper along a straight 3D line with a constant linear velocity without changing the orientation of the gripper. The concept of this controller is illustrated in Figure 3 for a 2D robot arm.

At each iteration of the trajectory, the error vector $\boldsymbol{e}$ is computed. This vector is obtained by computing the coordinates of the goal point P *in the gripper frame* with ${}^{G}P = {}^{G}E_W\,{}^{W}P$, where ${}^{G}E_W$ is obtained from the inverse of the forward kinematics (Section 4).

We use this vector $\boldsymbol{e}$ as a ***body*** linear velocity ${}^{W}v_G^b$ of the gripper. That is, $\boldsymbol{e}$ defines the direction we want to move the gripper as seen from the gripper's coordinate frame G.

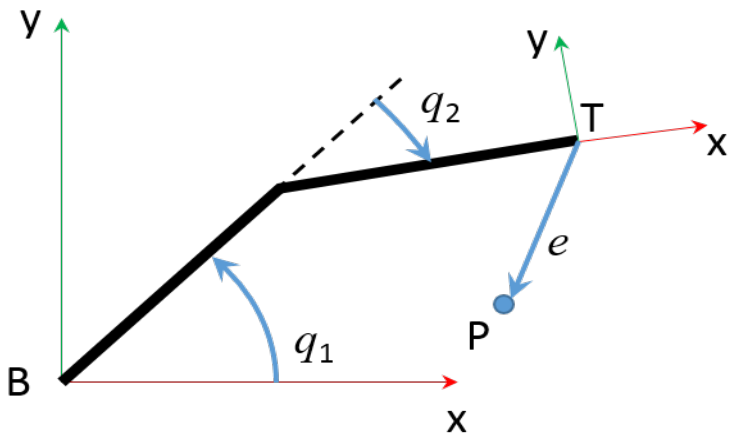

Figure 3. Resolve motion rate control

During lectures, the manipulator Jacobian was introduced to transform joint velocities to ***spatial*** velocity

$$\begin{bmatrix} {}^{W}v_G^s \\ {}^{W}\omega_G^s \end{bmatrix} = J(\boldsymbol{q})\dot{\boldsymbol{q}},$$

likewise, the 6x6 adjoint transformation was introduced to transform ***body*** velocity to ***spatial*** velocity with

$$\begin{bmatrix} {}^{W}v_G^s \\ {}^{W}\omega_G^s \end{bmatrix} = Ad \begin{bmatrix} {}^{W}v_G^b \\ {}^{W}\omega_G^b \end{bmatrix} = \begin{bmatrix} R & \hat{t}R \\ 0 & R \end{bmatrix} \begin{bmatrix} {}^{W}v_G^b \\ {}^{W}\omega_G^b \end{bmatrix}$$

where $\hat{t}$ is the skew-symmetric matrix of the translation t. Since $\boldsymbol{e}$ represents the desired ***body*** linear velocity ${}^{W}v_G^b$, we can write

$$\begin{bmatrix} {}^{W}v_G^b \\ {}^{W}\omega_G^b \end{bmatrix} = Ad^{-1}J(\boldsymbol{q})\dot{\boldsymbol{q}}.$$

Using the body velocity ${}^{W}v_G^b$, your generator will use the current robot joint angles $\boldsymbol{q}$ to

1) Evaluate the spatial Jacobian $J(\boldsymbol{q})$.
2) Multiply $J(\boldsymbol{q})$ by Ad^{-1}.
3) Invert $Ad^{-1}J(\boldsymbol{q})$ to solve the joint velocities $\dot{\boldsymbol{q}}$.

Your parts of this work are to derive A^{-1} and the 6x6 manipulator spatial Jacobian $J(\boldsymbol{q})$. The rest of the work (solving for $\dot{\boldsymbol{q}}$) is done. Once the vector $\dot{\boldsymbol{q}}$ is obtained, the robot will move its joints accordingly for a small time interval (say for 0.01 second) and the process will repeat again until the gripper reaches point P.

I highly recommend that you use Matlab to derive the equations and derivatives and export the equations with the command "ccode".

6. Implementation of the Trajectory

A trajectory node is partially implemented in assignment1/src/trajectory.cpp. The idea is for the trajectory node to publish messages that are identical to those from the GUI. It is thus important to switch off the GUI (the red button) when using your trajectory generator otherwise the GUI will command the robot to remain in the position specified by the sliders. The trajectory node subscribes to the topic `/setpoint` that provides the goal 3D coordinates of the gripper. Then, using the command `ros2 topic pub` you will publish the goal 3D coordinates of the Gripper and the trajectory node will interpolate the position of the gripper between its current 3D coordinates to the goal. For example, the command

```
$ ros2 topic pub -1 /setpoint geometry_msgs/msg/Point '{x: 0.5, y: 0.2,
z: 0.5}'
```

will move the gripper from its current position to the 3D coordinates "0.5 0.2 0.5" (these values are for illustration purposes you will need to adjust these coordinates based on the current position of your robot).

Your task is to finish the implementation of the trajectory generation. In the file assignment1.cpp, you will find several TODO comments. You will need to fix the code to make the trajectory node work. Essentially, you need to implement the functions in the file assignment1.cpp

1) ForwardKinematics (3 pt). To know the current grippers position.
2) ForwardKinematicsInverse (1pt). To know the coordinates of P in the gripper's frame.
3) AdjointTransformationInverse (2pt). To convert spatial velocities to body velocities.
4) Jacobian (3pt). To convert Cartesian velocities to joint velocities.

For 1) and 2) you are given $q_1, \ldots, q_6$ and you are asked to fill a 4x4 homogeneous transformation that contains the 3D position and orientation of the gripper with respect to the world coordinate frame (and its inverse). For 3) and 4), you need to implement the matrix Ad^{-1} that converts spatial velocities to body velocities and you need to implement the 6x6 manipulator Jacobian for this robot.

Be careful to fill in all the elements of these matrices, ***do not assume that all values are initially set to "0"***. ***I repeat: make sure you set all the elements of the all the matrices in your code.***

You can build your implementation by typing

```
$ colcon build
$ source install/setup.bash
```

from your `sbr_workspace` directory. For each terminal you use you always need to "source" the file install/setup.bash in your workspace (this is the kind of details you learn by doing the ROS tutorials).

Hints:

1. Start with simple small motions from a good initial configuration. Don't swing for a homerun on the first pitch. Make your code work on simple cases. These are easier to troubleshoot.
2. Whenever you want to move the robot to a desired x-y-z. It is essential to start the robot in a non-singular configuration. That is, a configuration where the Jacobian is ***not*** singular. A good example of this is:

Joint Name	Angle
shoulder_pan_joint	0.0
shoulder_lift_joint	-1.5708
elbow_joint	-1.5708
wrist_1_joint	-1.5708
wrist_2_joint	1.5708
wrist_2_joint	0.0

Try this initial configuration before experimenting with other initial configurations.

3. The robot may cross a singular configuration during a trajectory (messages should be printed on the terminal when that happens). You cannot avoid these so the best way to prevent them is to perform relatively small motions (10cm) from a good non-singular initial configuration. When that happens, the robot will not move. Shutdown everything and start again.
4. Check that the gripper reaches the coordinates it is supposed to go to (`tf2_echo`).

7. Deliverables

You need to create and submit your assignment1 package by using the following command. One (1) mark will be deducted if your package is not created as follow and one (1) mark will be deducted for packages that do not build.

```
$ tar cvfz ~/assignment1.tgz -C ~/sbr_workspace/src/sbr assignment1
```

This will create the file assignment1.tgz in your home directory. You must submit this file on Canvas. Double, ***triple*** check that your code is in the file you intend to submit (open assignment1.tgz file to inspect the files in there).

Reminder: By submitting your assignment, you acknowledge that you have read the JHU Academic Misconduct Policy and that you are the author of 100% of the code submitted. You are not allowed to use code completion and automatic programming tools. Furthermore, distributing or sharing your assignments solution or part thereof (whereas online, with classmates or students that will take this course in future semesters) is prohibited. Any academic misconduct, big or small, will be sanctioned with a course grade “F”. No negotiation. You have been warned.

A 10% penalty per day for late submission is strictly enforced.